ScopePilot — Project Module Implementation Report

Module: Project & Proposal Management | Author: Abhtej | Active Branch: Abhis

Project Platform	ScopePilot — AI-Powered Freelance Scope & Change Management Platform
Module Ownership	Project & Proposal Management (Abhtej)
Implementation Target	Project Module: Core Model, Routes, Service, Controller, Validation & Security
Active Git Branch	Abhis (current tracking branch)
Test Suite Result	19/19 Unit & Integration Tests Passed (100% Pass Rate across 5 test suites)
Module Status	Project Module Complete & Fully Verified
Report Date	October 15, 2026

1. Summary of Files Modified & Created

In strict compliance with the architecture and findings from Stage 1, the Project module was implemented without altering or duplicating teammate-owned code (Auth, Workspace, Milestones, Deliverables, Change Requests, or AI):

File Path	Action	Exact Enhancements & Responsibilities
backend/src/modules/project/project.model.js	**Enhanced**	Added virtual getters for workspaceId, freelancerId, and clientId with { virtuals: true } for full API payload compatibility while maintaining Mongoose ObjectId ref relations.
backend/src/modules/project/project.validation.js	**Enhanced**	Enhanced createProjectSchema to support both workspace/workspaceId and client/clientId aliases; added listProjectsQuerySchema.
backend/src/modules/project/project.service.js	**Enhanced**	Enforced strict workspace existence, creator membership checks, client existence & workspace membership verification, safe populated access control on retrieval, and implemented listProjects().
backend/src/modules/project/project.controller.js	**Enhanced**	Added listProjects handler delegating to service and returning standardized ApiResponse envelopes.
backend/src/modules/project/project.routes.js	**Enhanced**	Mounted GET / (list projects with query validation) alongside POST / and GET /:id, all protected by requireAuth.
backend/tests/project.test.js	**Created**	New comprehensive test suite containing 19 automated tests covering authentication guards, validation, workspace boundaries, project creation, retrieval access control, and scoped listing.

2. Project Model & Relational Architecture

The Project entity represents an authoritative freelance engagement. It establishes the central bridge connecting the workspace, the executing freelancer, and the hiring client:

workspace: ObjectId reference (ref: 'Workspace', required, indexed). Establishes multi-tenant tenant isolation under Sushant's Workspace module. Accessible via project.workspace and virtual project.workspaceId.

freelancer: ObjectId reference (ref: 'User', required, indexed). Represents the authenticated freelancer executing the work. Automatically populated from req.user._id upon creation. Accessible via project.freelancer and virtual project.freelancerId.

client: ObjectId reference (ref: 'User', optional, indexed). Represents the hiring client. When supplied, ProjectService verifies the client exists and is a valid member of the parent workspace. Accessible via project.client and virtual project.clientId.

title: String (required, trimmed, min 2, max 200). Human-readable title of the project engagement.

description: String (optional, default empty). Detailed overview of the project objectives.

status: String enum ('draft', 'active', 'in_progress', 'completed', 'cancelled', default: 'draft', indexed). Tracks the engagement lifecycle.

currentApprovedProposal: ObjectId reference (ref: 'ProposalVersion'). Reserved for Stage 3 to link the active, client-approved scope baseline for downstream Milestone and Change Request audit.

timestamps: Automatically managed by Mongoose (createdAt, updatedAt).

3. Implemented Endpoints & API Specifications

All endpoints are mounted under the /api/projects prefix and require a valid Bearer JWT token:

Endpoint 1: POST /api/projects — Create a Project

Creates a new project within a workspace. Freelancer is automatically bound from the authenticated JWT session (req.user._id). The creator and specified client are verified against the parent workspace membership.

Example Success Response (201 Created)


```
{
  "success": true,
  "message": "Project created successfully",
  "data": {
    "_id": "670d8a9999999999abcdef99",
    "workspace": "670d8a1234567890abcdef01",
    "workspaceId": "670d8a1234567890abcdef01",
    "freelancer": "670d8a1234567890abcdef00",
    "freelancerId": "670d8a1234567890abcdef00",
    "client": "670d8a1234567890abcdef02",
    "clientId": "670d8a1234567890abcdef02",
    "title": "E-Commerce Headless Storefront",
    "description": "Full-stack Next.js storefront with Stripe checkout integration.",
    "status": "draft",
    "createdAt": "2026-10-15T00:08:00.000Z",
    "updatedAt": "2026-10-15T00:08:00.000Z"
  }
}
```


Endpoint 2: GET /api/projects/:id — Retrieve Project Details

Retrieves detailed information for a specific project. Populates workspace, freelancer, client, and approved proposal version. Enforces strict access control: only the assigned freelancer, assigned client, or members of the parent workspace are authorized.

Example Success Response (200 OK)


```
{
  "success": true,
  "message": "Project retrieved successfully",
  "data": {
    "_id": "670d8a9999999999abcdef99",
    "title": "E-Commerce Headless Storefront",
    "description": "Full-stack Next.js storefront with Stripe checkout integration.",
    "status": "draft",
```

```
  "workspace": {
    "_id": "670d8a1234567890abcdef01",
    "name": "Acme Digital Labs",
    "slug": "acme-digital-labs",
    "owner": "670d8a1234567890abcdef00"
  },
  "freelancer": {
    "_id": "670d8a1234567890abcdef00",
    "name": "Alex Freelancer",
    "email": "freelancer@test.com",
    "role": "freelancer"
  },
  "client": {
    "_id": "670d8a1234567890abcdef02",
    "name": "Sarah Client",
    "email": "client@test.com",
    "role": "client"
  },
  "currentApprovedProposal": null,
  "createdAt": "2026-10-15T00:08:00.000Z"
 }
}
```


Endpoint 3: GET /api/projects — List Accessible Projects

Lists all projects accessible to the authenticated user, supporting optional query filters workspaceId (or workspace) and status. Prevents unauthorized users from probing projects in workspaces they do not belong to.

4. Automated Test Results (100% Pass Rate)

A dedicated Jest/Supertest suite was executed. All 19 tests in the Project test suite and all 32 tests across the full backend passed without error:

Test Category	Scenarios Verified	Outcome
Authentication Guards	Missing Bearer token, invalid token signature, expired token.	**3/3 Passed (401 Unauthorized)**
Zod Input Validation	Missing workspace ID, invalid ObjectId hex format, title too short (<2 chars), invalid status enum, malformed :id parameter.	**5/5 Passed (400 Bad Request)**
Workspace Security & References	Non-existent workspace (404), creator not a workspace member (403), client user not in workspace (400).	**3/3 Passed (Boundary Enforced)**
Project Creation	Valid project creation with auto-assigned freelancer, workspaceId/clientId alias compatibility.	**2/2 Passed (201 Created)**
Project Retrieval & Access Control	Non-existent project (404), freelancer access (200), client access (200), cross-workspace intruder blocked (403).	**4/4 Passed (Access Scoped)**
Project Listing	Workspace member listing (200), outsider workspace query blocked (403).	**2/2 Passed (Listing Scoped)**

5. Step-by-Step Demonstration for Faculty Review

To demonstrate the functionality of the Project module during faculty project evaluation, follow these sequential steps:

Step 1: Start Server & Seed Database: Ensure Node.js (v18+) is installed. Run 'npm run seed' in backend/ to populate test users (freelancer@scopepilot.dev, client@scopepilot.dev, password123) and sample workspace 'Acme Digital Labs'.

Step 2: Authenticate as Freelancer: Send POST /api/auth/login with freelancer credentials to obtain JWT token. Store in environment variable as FREELANCER_TOKEN.

Step 3: Demonstrate Validation Error Handling: Send POST /api/projects with Authorization: Bearer <FREELANCER_TOKEN> and invalid body (e.g. missing workspace or 1-character title). Show the faculty the 400 Bad Request response with standardized Zod error fields.

Step 4: Demonstrate Successful Project Creation: Send POST /api/projects with valid workspace ID, client ID, title, and description. Demonstrate the 201 Created response showing auto-assigned freelancer and virtual workspaceId/clientId.

Step 5: Demonstrate Authorized Retrieval: Send GET /api/projects/:id using the newly created ID with freelancer token. Demonstrate the 200 OK populated response.

Step 6: Demonstrate Access Security: Authenticate as an unrelated client from another workspace, then attempt GET /api/projects/:id on the project. Demonstrate the 403 Forbidden response proving cross-workspace tenant isolation.

Step 7: Run Automated Test Suite: Run 'npm test -- tests/project.test.js' in the terminal to show all 19 automated tests passing live.

6. Explicit Boundaries & Next Steps (Stage 3)

In strict accordance with project phase restrictions, the following downstream features were intentionally omitted and will be implemented in Stage 3 upon your review:

Proposal Module (Stage 3): Multi-version proposal authoring, version bumping (v1 -> v2), cryptographic SHA-256 contentHash generation, and client-only approval enforcement.

Milestones & Deliverables: Milestone progress tracking, deliverables submission, and scopeReference audit linking (Mounish's ownership).

Change Requests: Change request submission, impact assessment, and semantic comparison (Mokshagna's ownership).

AI Module: LangChain agents, vector embeddings, and Atlas Vector Search (Vishal's ownership).